Lab-11: Write Java Program and JUnit Test Cases in Jenkins

Objective:
Create a simple Java application, write JUnit test cases, integrate with Maven, and execute tests through a Jenkins pipeline.

Lab Architecture
You will:

1. Create a simple Java Maven project.
2. Write a Java class.
3. Write JUnit test cases.
4. Push code to Git.
5. Configure Jenkins.
6. Run build & tests via Jenkins.
7. Publish test reports.

Prerequisites
Installed on machine/server:

- Java 17+
- Maven
- Git
- Jenkins (running)
- Browser access to Jenkins

Verify locally:
java -version
mvn -version
git --version

Step 1 – Create Maven Project
Option A (Recommended – Maven Archetype)
Run:
mvn archetype:generate
Choose:

- GroupId: com.devops.lab
- ArtifactId: lab11-junit
- Packaging: jar
- Archetype: maven-archetype-quickstart

Move into project:
cd lab11-junit

Step 2 – Project Structure Review

You should see:

```
src/
 ├── main/java/
 └── test/java/
pom.xml
```

This structure is critical because Maven automatically runs tests inside:

src/test/java

Step 3 – Write Java Program

Navigate to:

src/main/java/com/devops/lab/

Create:

Calculator.java

```
package com.devops.lab;

public class Calculator {

  public int add(int a, int b) {
    return a + b;
  }

  public int divide(int a, int b) {
    if(b == 0) {
      throw new IllegalArgumentException("Cannot divide by zero");
    }
    return a / b;
  }
}
```

Step 4 – Add JUnit Dependency in pom.xml

Open pom.xml.

Replace dependency section with:

```
<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.10.0</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

```
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>3.1.2</version>
    </plugin>
  </plugins>
</build>
```

Explanation:

- JUnit 5 for testing.
- Surefire plugin executes test cases.

Step 5 – Write JUnit Test Cases

Navigate to:

src/test/java/com/devops/lab/

Create:

CalculatorTest.java

```
package com.devops.lab;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class CalculatorTest {

  Calculator calculator = new Calculator();

  @Test
  void testAddition() {
    assertEquals(10, calculator.add(5,5));
  }

  @Test
  void testDivision() {
    assertEquals(2, calculator.divide(10,5));
  }

  @Test
  void testDivideByZero() {
    Exception exception = assertThrows(
      IllegalArgumentException.class,
```

```
        () -> calculator.divide(10,0)
    );

    assertEquals("Cannot divide by zero", exception.getMessage());
  }
}
```

Step 6 – Run Tests Locally

Execute:

mvn clean test

Expected:

- BUILD SUCCESS
- Tests run: 3
- Failures: 0

Check report:

target/surefire-reports

Step 7 – Push Code to Git Repository

Initialize Git:

```
git init
git add .
git commit -m "Initial commit with Java app and JUnit tests"
```

Create GitHub repo and push:

```
git remote add origin <repo-url>
git branch -M main
git push -u origin main
```

Step 8 – Configure Jenkins

Open Jenkins:

http://localhost:8080

Step 8.1 – Install Required Plugins

Go to:

Manage Jenkins → Plugins

Ensure installed:

- Git Plugin
- Maven Integration Plugin
- Pipeline Plugin
- JUnit Plugin

Restart if required.

Step 8.2 – Configure Maven & JDK
Go to:
Manage Jenkins → Global Tool Configuration
Add:
JDK:

- Name: JDK17
- Path to Java home

Maven:

- Name: Maven3
- Install automatically OR provide local path

Save.

Step 9 – Create Jenkins Pipeline Job

1. Click "New Item"
2. Enter: Lab-11-Java-JUnit
3. Select: Pipeline
4. Click OK

Step 10 – Add Jenkinsfile
Inside project root, create:
Jenkinsfile

```
pipeline {
  agent any

  tools {
    jdk 'JDK17'
    maven 'Maven3'
  }

  stages {

    stage('Checkout') {
      steps {
        git '<your-repo-url>'
      }
    }

    stage('Build') {
      steps {
        sh 'mvn clean compile'
      }
```

```
    }

    stage('Test') {
      steps {
        sh 'mvn test'
      }
    }
  }

  post {
    always {
      junit '**/target/surefire-reports/*.xml'
    }
  }
}
```

Commit and push Jenkinsfile.

Step 11 – Configure Pipeline Source

In Jenkins job:

Pipeline → Definition → Pipeline script from SCM

SCM: Git

Repository URL: <your-repo-url>

Branch: main

Script Path: Jenkinsfile

Save.

Step 12 – Run Build

Click:

Build Now

Observe console output:

- Checkout stage
- Build stage
- Test stage

Expected:

BUILD SUCCESS

Step 13 – View Test Reports in Jenkins

After build:

Click build number → Test Result

You should see:

- 3 tests

- 0 failures

Jenkins parses Surefire XML reports automatically.

Step 14 – Validate Failure Scenario (Important)

Modify Calculator:

Change:

return a + b;

To:

return a + b + 1;

Push change.

Re-run pipeline.

Expected:

- Test failure
- Build marked FAILED

This demonstrates:

✓ Fail-fast principle

✓ CI quality gate

✓ Automated regression safety

Lab Completion Checklist

✅ Java project created

✅ JUnit tests written

✅ Tests executed locally

✅ Code pushed to Git

✅ Jenkins pipeline created

✅ Test reports published

✅ Failure scenario validated

Learning Outcomes

Participants now understand:

- Maven project lifecycle
- JUnit test execution
- Surefire test reporting
- Jenkins pipeline automation
- CI fail-fast mechanism
- Automated regression validation